DESIGN AND ANALYSIS OF ALGORITHMS

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

COURSE CODE & NAME:

CSA06 – DESIGN AND ANALYSIS OF ALGORITHMS

ASSESSMENT CO3-AT2 – CASE STUDY

DIVIDE-AND-CONQUER ENGINE OF A RETAIL DATA ANALYTICS PLATFORM

Case Study Title:

Design and Analysis of Divide-and-Conquer Algorithms for KartIQ Retail Data Analytics Platform

Submitted by: S.Harshavardhan

Registration Number:192524106

B.Tech – Artificial Intelligence and Data Science

SIMATS Engineering

1. INTRODUCTION

Divide and Conquer is an important algorithm design technique used to solve large computational problems efficiently. The main idea is to divide a large problem into smaller subproblems, solve each subproblem independently, and combine their results to obtain the final solution.

The three main steps of the Divide and Conquer technique are:

1. **Divide:** Break the original problem into smaller subproblems.
2. **Conquer:** Solve the smaller subproblems, usually through recursion.
3. **Combine:** Combine the results of the subproblems to produce the final answer.

This technique is widely used in sorting, searching, and matrix multiplication. Some important algorithms based on Divide and Conquer are Merge Sort, Quick Sort, Binary Search, and Strassen's Matrix Multiplication.

2. CASE STUDY OVERVIEW

KartIQ is a retail data analytics platform used by a national e-commerce marketplace. It processes approximately 10,000,000 orders every day and manages the performance rankings of 100,000 sellers.

The platform currently faces several performance problems:

- The nightly order report takes too much time to complete.
- The live seller leaderboard freezes during major sale events.
- Customer-support staff experience delays while searching for orders.
- Product recommendation scores are updated only once a day.

These problems affect customers, sellers, and auditors. Therefore, the platform requires efficient algorithms to process large amounts of data within a short period.

As an algorithm engineer, the objective is to redesign the four major modules using suitable Divide and Conquer algorithms.

3. OBJECTIVES OF THE CASE STUDY

The main objectives of this case study are:

1. To understand the working principles of Divide and Conquer algorithms.
2. To apply Merge Sort for sorting large numbers of orders by timestamp.
3. To analyse Quick Sort and improve its pivot selection strategy.

4. To apply Binary Search for faster order lookup.
5. To understand Strassen's Matrix Multiplication for calculating recommendation scores.
6. To analyse the time and space complexity of each algorithm.
7. To compare different algorithmic approaches and select the most suitable solution for the KartIQ platform.

4. MODULES IDENTIFIED

The KartIQ platform contains four important computational modules.

Module 1 – Nightly Order Consolidation: Merge Sort is used to arrange all orders according to their timestamps while maintaining the original arrival order of orders having equal timestamps.

Module 2 – Live Seller Leaderboard: Quick Sort is used to arrange seller scores. An improved pivot selection method is required to avoid poor performance when the scores are nearly sorted.

Module 3 – Order Lookup: Binary Search is used to find orders quickly in the sorted order list. The modified algorithm must return the first occurrence when multiple orders have the same timestamp.

Module 4 – Recommendation Scores: Strassen's Matrix Multiplication is used to calculate product recommendation scores by multiplying two matrices efficiently.

5. IMPORTANCE OF THE STUDY

The study helps identify suitable algorithms for handling large-scale retail data. Selecting the correct algorithm improves processing speed, reduces unnecessary operations, and makes the platform more reliable.

By applying Divide and Conquer techniques, KartIQ can improve its order processing, seller ranking, order searching, and recommendation calculations.

2 – (a) UNDERSTANDING THE CASE

6. COMPUTATIONAL REQUIREMENTS AND KEY ISSUES

The KartIQ platform contains four major modules. Each module has a different computational requirement and requires a suitable algorithm to achieve better performance.

The main requirements, key issues, and consequences of choosing a poor algorithm are explained below.

6.1 MODULE-WISE ANALYSIS TABLE

Module	Computational Requirement	Key Issues	Suitable Algorithm
Module 1 – Nightly Order Consolidation	Sort 10,000,000 orders based on timestamp.	Stability, large input size, memory usage, and equal timestamps.	Merge Sort
Module 2 – Live Seller Leaderboard	Sort 100,000 seller scores repeatedly.	Nearly sorted data, pivot selection, and worst-case behaviour.	Improved Quick Sort
Module 3 – Order Lookup	Find an order using its timestamp.	Sorted-data requirement and duplicate timestamps.	Binary Search
Module 4 – Recommendation Scores	Multiply two 1024 × 1024 matrices.	Large matrix size, number of multiplications, and memory usage.	Strassen's Method

6.2 MODULE 1 – NIGHTLY ORDER CONSOLIDATION

Computational Requirement:

The platform receives approximately 10,000,000 orders every day. These orders must be sorted according to their timestamps to generate the nightly audit report.

Key Issues:

1. The sorting algorithm must handle a very large number of orders.
2. It must maintain stability.
3. Orders having the same timestamp must preserve their original arrival order.
4. The algorithm must use memory efficiently.
5. The sorted output must be suitable for further searching.

Consequence of Choosing a Poor Algorithm:

If an inefficient sorting algorithm such as basic Bubble Sort is used, the execution time will become extremely high for 10,000,000 orders. If an unstable sorting algorithm is selected, orders with equal timestamps may change their original arrival order, creating problems during gateway reconciliation.

Selected Algorithm: Merge Sort because it provides O(n log n) time complexity and is stable when implemented correctly.

6.3 MODULE 2 – LIVE SELLER LEADERBOARD

Computational Requirement:

The platform maintains the scores of 100,000 sellers. These scores are sorted repeatedly to display the live seller leaderboard.

Key Issues:

1. Seller scores change only slightly between refreshes.
2. Therefore, the data is usually nearly sorted.
3. The current Quick Sort implementation selects the first element as the pivot.
4. A poor pivot can create highly unbalanced partitions.
5. The leaderboard must respond quickly during major sale events.

Consequence of Choosing a Poor Algorithm:

If the first-element pivot is used on nearly sorted data, Quick Sort may repeatedly divide the array into one small part and one very large part. This produces $O(n^2)$ worst-case performance and can cause the leaderboard to freeze.

Selected Algorithm: Improved Quick Sort using median-of-three or randomized pivot selection. An introspective sorting approach can also be used for additional worst-case protection.

6.4 MODULE 3 – ORDER LOOKUP

Computational Requirement:

Customer-support staff need to search for orders using their timestamps in the consolidated order list.

Key Issues:

1. The list must already be sorted.
2. Multiple orders may have the same timestamp.
3. Staff need the first order having the requested timestamp.
4. Searching must be fast even when millions of orders are stored.

Consequence of Choosing a Poor Algorithm:

Using Linear Search may require checking every order. For 10,000,000 orders, this can result in millions of comparisons for a single search.

Selected Algorithm: Modified Binary Search because it reduces the search range by half at every step and can find the first occurrence of a timestamp.

6.5 MODULE 4 – RECOMMENDATION SCORES

Computational Requirement:

The platform calculates recommendation scores by multiplying the segment-feature preference matrix A with the feature-product matrix B.

The formula is:

$C = A \times B$

In production, both matrices are of size 1024×1024.

Key Issues:

1. Matrix multiplication requires a large number of arithmetic operations.
2. The matrices are large.
3. Processing time affects recommendation refresh frequency.
4. Memory usage must be considered.
5. The result must be mathematically correct.

Consequence of Choosing a Poor Algorithm:

Conventional matrix multiplication requires $O(n^3)$ time. For large matrices, this can increase processing time and delay recommendation updates.

Selected Algorithm: Strassen's Matrix Multiplication because it reduces the number of recursive matrix multiplications from eight to seven.

7. OVERALL ANALYSIS

The four modules require different algorithms because their computational requirements are different. Merge Sort is suitable for stable sorting, improved Quick Sort is suitable for repeated leaderboard sorting, Binary Search is suitable for searching sorted data, and Strassen's Method is suitable for reducing matrix multiplication operations.

Selecting the correct algorithm helps KartIQ improve performance, reduce delays, and provide faster services to customers and sellers.

3 – (b) MODULE 1: MERGE SORT FOR NIGHTLY ORDER CONSOLIDATION

8. INTRODUCTION TO MERGE SORT

Merge Sort is a sorting algorithm based on the Divide and Conquer technique. It divides the input array into two smaller halves, sorts each half recursively, and merges the sorted halves into one final sorted array.

Merge Sort is suitable for the KartIQ platform because it can efficiently sort millions of orders and maintain the original arrival order of orders having equal timestamps.

Time Complexity: O(n log n)

Space Complexity: O(n)

Stability: Stable

8.1 WORKING PRINCIPLE OF MERGE SORT

Merge Sort works using three main steps.

1. Divide:

The given list is divided into two approximately equal halves until each sublist contains only one element.

2. Conquer:

Each smaller sublist is sorted recursively. A single-element list is already sorted.

3. Combine:

The sorted sublists are merged by comparing their elements and arranging them in ascending order.

8.2 GIVEN SAMPLE INPUT

The sample orders provided in the case study are:

Timestamp	Order ID
105	O1
102	O2
105	O3
101	O4
102	O5
103	O6

Initial Input:

[(105, O1), (102, O2), (105, O3), (101, O4), (102, O5), (103, O6)]

The objective is to sort the orders in ascending order of timestamp.

8.3 DIVIDE STEP

First, divide the original list into two halves.

Original List:

[(105, O1), (102, O2), (105, O3), (101, O4), (102, O5), (103, O6)]

First Division:

Left Half:

[(105, O1), (102, O2), (105, O3)]

Right Half:

[(101, O4), (102, O5), (103, O6)]

Now divide both halves again.

Left Half Division:

[(105, O1)]

[(102, O2), (105, O3)]

Further division:

[(102, O2)]

[(105, O3)]

Right Half Division:

[(101, O4)]

[(102, O5), (103, O6)]

Further division:

[(102, O5)]

[(103, O6)]

At this stage, every sublist contains only one element or is ready for merging.

8.4 CONQUER AND COMBINE STEP

Now merge the smaller lists in ascending order of timestamp.

Merge 1:

Compare:

[(102, O2)] and [(105, O3)]

Since 102 is smaller than 105, the result is:

[(102, O2), (105, O3)]

Merge 2:

Compare:

[(105, O1)] and [(102, O2), (105, O3)]

The timestamp 102 is smaller, so O2 is placed first. Then O1 is placed before O3 because both have timestamp 105 and O1 appeared earlier in the original input.

Result:

[(102, O2), (105, O1), (105, O3)]

Merge 3:

Compare:

[(102, O5)] and [(103, O6)]

Since 102 is smaller than 103, the result is:

[(102, O5), (103, O6)]

Merge 4:

Compare:

[(101, O4)] and [(102, O5), (103, O6)]

The result is:

[(101, O4), (102, O5), (103, O6)]

Final Merge:

Merge the two sorted halves:

Left:

[(102, O2), (105, O1), (105, O3)]

Right:

[(101, O4), (102, O5), (103, O6)]

Final sorted output:

[(101, O4), (102, O2), (102, O5), (103, O6), (105, O1), (105, O3)]

8.5 FINAL RESULT

Timestamp	Order ID
101	O4
102	O2
102	O5
103	O6
105	O1

105	O3

The orders are now arranged in ascending order of timestamp.

PAGE 4 – MERGE SORT STABILITY AND COMPLEXITY ANALYSIS

8.6 STABILITY OF MERGE SORT

Stability means that if two elements have the same sorting key, their original relative order must be maintained after sorting.

In the KartIQ platform, orders O1 and O3 both have timestamp 105. In the original input, O1 appears before O3.

Similarly, orders O2 and O5 both have timestamp 102. In the original input, O2 appears before O5.

After applying Merge Sort, the output is:

[(101, O4), (102, O2), (102, O5), (103, O6), (105, O1), (105, O3)]

We can observe that:

- O2 appears before O5.
- O1 appears before O3.

Therefore, the original arrival order of orders having equal timestamps is maintained.

Important: To maintain stability, the merge operation must select the element from the left subarray when both timestamps are equal. This ensures that the earlier element remains before the later element.

8.7 RECURRENCE RELATION OF MERGE SORT

Merge Sort divides the input into two equal parts and recursively sorts both parts.

After sorting, the two halves are merged. The merging operation takes linear time, represented by O(n).

The recurrence relation is:

$T(n) = 2T(n/2) + cn$

Where:

- T(n) represents the total running time.
- 2T(n/2) represents the time required to sort two halves.
- cn represents the time required to merge the two sorted halves.
- c is a constant.
- n is the number of elements.

The base condition is:

$T(1) = c$

When only one element is present, no sorting operation is required.

8.8 SOLUTION USING MASTER THEOREM

The general form of the Master Theorem is:

$T(n) = aT(n/b) + f(n)$

For Merge Sort:

$a = 2$

$b = 2$

$f(n) = cn$

Calculate:

$n^{(\log_b a)} = n^{(\log_2 2)}$

$= n^1$

$= n$

Since $f(n) = \Theta(n)$, both terms have the same growth rate.

According to Case 2 of the Master Theorem:

$T(n) = \Theta(n \log n)$

Therefore, the time complexity of Merge Sort is:

Best Case: O(n log n)

Average Case: O(n log n)

Worst Case: O(n log n)

8.9 COMPARISON ESTIMATE FOR 10,000,000 ORDERS

The KartIQ platform processes approximately 10,000,000 orders every day.

Let:

$n = 10,000,000$

The approximate number of comparison operations for Merge Sort is proportional to:

$n \log_2 n$

$\log_2(10,000,000) \approx 23.2535$

Therefore:

$n \log_2 n = 10{,}000{,}000 \times 23.2535$

$\approx 232{,}535{,}000$

Hence, the rough comparison estimate is approximately 232.5 million comparison units.

The actual number of comparisons depends on the input arrangement and merge operations. This value is an asymptotic estimate, not an exact comparison count.

This demonstrates that Merge Sort can handle large datasets much more efficiently than simple $O(n^2)$ sorting algorithms.

8.10 MEMORY REQUIREMENT

Merge Sort requires additional memory to store temporary arrays during the merging process.

Space Complexity: $O(n)$

For 10,000,000 orders, the algorithm may require a significant amount of additional memory.

The platform must consider the following:

1. Temporary arrays are needed during merging.
2. The memory requirement increases with the number of orders.
3. Additional memory is required to store the original order records.
4. The implementation should avoid unnecessary copying of large order objects.

If the complete dataset does not fit into RAM, the platform can use External Merge Sort.

8.11 EXTERNAL MERGE SORT

External Merge Sort is used when the input data is too large to fit into main memory.

The process works as follows:

1. Divide the large order dataset into smaller chunks.
2. Load one chunk into RAM.
3. Sort each chunk using Merge Sort.
4. Store the sorted chunks in secondary storage.
5. Merge the sorted chunks using a multiway merge.
6. Continue merging until one completely sorted file is produced.

This method allows KartIQ to sort millions of orders even when the complete dataset cannot fit into RAM.

8.12 CONCLUSION OF MODULE 1

Merge Sort is the most suitable algorithm for the nightly order consolidation module because it provides O(n log n) time complexity and maintains stability. It preserves the original arrival order of orders having equal timestamps.

For extremely large datasets, External Merge Sort can be used to reduce memory limitations. Therefore, Merge Sort provides an efficient and reliable solution for KartIQ's nightly order processing.

PAGE 5 – (c) MODULE 2: QUICK SORT FOR THE LIVE SELLER LEADERBOARD

9. INTRODUCTION TO QUICK SORT

Quick Sort is a sorting algorithm based on the Divide and Conquer technique. It selects one element as a pivot and rearranges the remaining elements so that smaller elements are placed on one side and larger elements are placed on the other side.

After partitioning, Quick Sort recursively sorts the two partitions.

Quick Sort is widely used because of its good average-case performance and efficient in-place partitioning.

Best Case Time Complexity: O(n log n)

Average Case Time Complexity: O(n log n)

Worst Case Time Complexity: $O(n^2)$

Auxiliary Space Complexity: O(log n) on average for recursive calls; O(n) in the worst case.

9.1 GIVEN SAMPLE INPUT

The seller scores provided in the case study are:

42, 17, 63, 8, 55, 29

The objective is to arrange the seller scores in ascending order.

Initial Array:

[42, 17, 63, 8, 55, 29]

9.2 PARTITION STEP USING THE FIRST ELEMENT AS PIVOT

In the current implementation, the first element is selected as the pivot.

Pivot = 42

The array is:

[42, 17, 63, 8, 55, 29]

Now compare the remaining elements with the pivot.

- 17 is smaller than 42, so it belongs to the left partition.

- 63 is greater than 42, so it belongs to the right partition.
- 8 is smaller than 42, so it belongs to the left partition.
- 55 is greater than 42, so it belongs to the right partition.
- 29 is smaller than 42, so it belongs to the left partition.

After partitioning, one possible arrangement is:

[17, 8, 29, 42, 55, 63]

Here:

Left partition = [17, 8, 29]

Pivot = 42

Right partition = [55, 63]

9.3 RECURSIVE SORTING

Now Quick Sort is applied recursively to both partitions.

Left partition:

[17, 8, 29]

After sorting:

[8, 17, 29]

Right partition:

[55, 63]

After sorting:

[55, 63]

Finally, combine the sorted left partition, pivot, and right partition.

Final sorted array:

[8, 17, 29, 42, 55, 63]

9.4 WHY THE FIRST-ELEMENT PIVOT MAKES QUICK SORT SLOW

The current implementation selects the first element as the pivot for every partition.

This approach can become inefficient when the input data is already sorted or nearly sorted.

For example:

[8, 17, 29, 42, 55, 63]

If the first element is selected as the pivot, it is the smallest element.

Therefore, the partition produces:

Left partition = []

Right partition = [17, 29, 42, 55, 63]

The next recursive call again selects the first element of the remaining array as the pivot.

This process continues repeatedly, creating highly unbalanced partitions.

As a result, Quick Sort performs many unnecessary comparisons and its execution time increases.

For a large leaderboard containing 100,000 sellers, this problem can make the platform slow during major sale events.

9.5 BEST-CASE RECURRENCE

The best case occurs when the pivot divides the array into two nearly equal halves.

The recurrence relation is:

$T(n) = 2T(n/2) + cn$

Where:

- Two recursive calls sort the two halves.
- cn represents the partitioning work.

Using the Master Theorem:

$a = 2$

$b = 2$

$f(n) = cn$

Therefore:

$T(n) = \Theta(n \log n)$

Hence, the best-case time complexity of Quick Sort is:

O(n log n)

9.6 WORST-CASE RECURRENCE

The worst case occurs when the pivot is always the smallest or largest element.

In this situation, one partition contains zero elements and the other contains $n - 1$ elements.

The recurrence relation is:

$T(n) = T(n - 1) + cn$

Expanding:

$T(n) = T(n - 2) + c(n - 1) + cn$

Continuing:

$T(n) = T(1) + c(2 + 3 + ... + n)$

The sum of the first n natural numbers is:

$n(n + 1) / 2$

Therefore:

$T(n) = \Theta(n^2)$

Hence, the worst-case time complexity is:

$O(n^2)$

9.7 CONCLUSION OF QUICK SORT ANALYSIS

Quick Sort is efficient when the pivot divides the input into balanced partitions. However, selecting the first element as the pivot is unsuitable for nearly sorted seller scores.

An improved pivot selection strategy is required to reduce the possibility of unbalanced partitions and improve leaderboard performance.

PAGE 6 – QUICK SORT OPTIMISATION AND PERFORMANCE ANALYSIS

9.8 COMPARISON ESTIMATE FOR 100,000 SELLERS

The KartIQ platform maintains the scores of 100,000 sellers. These scores are nearly sorted because only a few seller scores change between two leaderboard refreshes.

Let:

$n = 100,000$

When the first element is selected as the pivot and the input is already sorted, Quick Sort produces highly unbalanced partitions.

The number of comparisons in the worst case is:

$C(n) = n(n - 1) / 2$

Substituting $n = 100,000$:

$C(n) = 100,000 \times 99,999 / 2$

$C(n) = 9,999,900,000 / 2$

$C(n) = 4,999,950,000$ comparisons

Therefore, the first-element pivot may require approximately 5 billion comparisons in the worst case.

This large number of comparisons can make the live leaderboard extremely slow during sale events.

For balanced partitions, the comparison growth is approximately proportional to $n \log_2 n$.

$\log_2(100{,}000) \approx 16.61$

Therefore:

$n \log_2 n = 100{,}000 \times 16.61$

$\approx 1{,}661{,}000$

This is an asymptotic comparison estimate for balanced partitioning, not an exact comparison count.

The comparison between the two situations shows why selecting a suitable pivot is important.

9.9 IMPROVED PIVOT SELECTION STRATEGY

To improve Quick Sort performance, the platform should avoid selecting the first element as the pivot every time.

Two suitable strategies are:

1. Median-of-Three Pivot Selection

In this method, the first, middle, and last elements are selected.

The median value among these three elements is used as the pivot.

For example, consider:

[8, 17, 29, 42, 55, 63]

First element = 8

Middle element = 29

Last element = 63

The median of these three values is 29.

Therefore:

Pivot = 29

This pivot is better than selecting 8 because it is more likely to divide the array into balanced partitions.

2. Randomized Pivot Selection

In this method, the pivot is selected randomly from the input array.

This reduces the possibility of repeatedly selecting the smallest or largest element.

Randomized pivot selection improves expected performance and makes the algorithm less dependent on the original input order.

9.10 RECOMMENDED APPROACH FOR KARTIQ

For the live seller leaderboard, median-of-three pivot selection is a suitable improvement over the current first-element pivot.

However, pivot selection alone does not guarantee O(n log n) worst-case performance.

For stronger protection, KartIQ can use Introsort. Introsort begins with Quick Sort and switches to Heap Sort if the recursion becomes too deep.

This provides:

- Good average-case performance.
- Better handling of nearly sorted data.
- Protection against Quick Sort's $O(n^2)$ worst case.
- Efficient sorting for a large number of sellers.

Since only a few seller scores change between refreshes, the platform can also consider insertion sort for very small partitions or incremental leaderboard updates.

9.11 CONCLUSION OF MODULE 2

The first-element pivot is unsuitable for the KartIQ leaderboard because nearly sorted data can produce highly unbalanced partitions.

Median-of-three or randomized pivot selection improves pivot quality. An Introsort-based implementation provides additional worst-case protection.

Therefore, an improved Quick Sort strategy is recommended for the live seller leaderboard.

PAGE 7 – (d) MODULE 3: BINARY SEARCH FOR ORDER LOOKUP

10. INTRODUCTION TO BINARY SEARCH

Binary Search is an efficient searching algorithm based on the Divide and Conquer technique. It works by repeatedly dividing a sorted array into two halves.

Instead of checking every element, Binary Search compares the target value with the middle element.

If the target is smaller, the search continues in the left half. If the target is larger, the search continues in the right half.

This process continues until the required element is found or the search range becomes empty.

Time Complexity: O(log n)

Space Complexity: O(1) for an iterative implementation.

10.1 IMPORTANCE OF SORTED DATA

Binary Search can only be applied correctly when the input data is sorted according to the search key.

In the KartIQ platform, Module 1 uses Merge Sort to arrange orders in ascending order of timestamp.

The sorted order list is:

[(101, O4), (102, O2), (102, O5), (103, O6), (105, O1), (105, O3)]

Since the timestamps are sorted, Binary Search can be applied to find a particular timestamp efficiently.

If the list is not sorted, Binary Search may produce an incorrect result because it decides which half to discard based on the ordering of elements.

Therefore, Merge Sort and Binary Search work together in the KartIQ platform.

10.2 BINARY SEARCH WORKING PRINCIPLE

Binary Search follows these steps:

1. Set the left boundary to the first element.
2. Set the right boundary to the last element.
3. Calculate the middle position.
4. Compare the middle timestamp with the target timestamp.
5. If both are equal, the element is found.
6. If the target is smaller, continue searching in the left half.
7. If the target is larger, continue searching in the right half.
8. Repeat until the element is found or the search range becomes empty.

10.3 GIVEN SAMPLE INPUT

Sorted timestamps:

[101, 102, 102, 103, 105, 105]

Target timestamp:

102

The objective is to find the first order having timestamp 102.

10.4 NORMAL BINARY SEARCH TRACE

Number the array positions starting from zero.

Index	Timestamp
0	101
1	102
2	102
3	103
4	105
5	105

Target = 102

Step 1:

Left = 0

Right = 5

Middle = (0 + 5) // 2 = 2

Middle timestamp = 102

The target is found at index 2.

However, there is another order with timestamp 102 at index 1.

Normal Binary Search may return index 2 instead of the first occurrence.

Therefore, a modified Binary Search is required.

10.5 MODIFIED BINARY SEARCH FOR FIRST OCCURRENCE

The modified Binary Search searches for the first occurrence of the target timestamp.

When the middle element matches the target, the algorithm does not stop immediately. Instead, it records the current position and continues searching in the left half.

This ensures that the first matching position is returned.

The algorithm works as follows:

1. Initialize left = 0 and right = n - 1.
2. Initialize result = -1.
3. Calculate the middle position.

4. If the middle timestamp equals the target, store the middle position in result.
5. Continue searching in the left half by updating right = mid - 1.
6. If the middle timestamp is smaller than the target, search the right half.
7. If the middle timestamp is greater than the target, search the left half.
8. Return result after the search is completed.

This method guarantees that the first occurrence is found.

PAGE 8 – BINARY SEARCH TRACE AND COMPLEXITY

10.6 TRACE OF MODIFIED BINARY SEARCH

Given sorted timestamps:

[101, 102, 102, 103, 105, 105]

Target = 102

Step 1:

Left = 0

Right = 5

Middle = (0 + 5) // 2 = 2

Timestamp at index 2 = 102

The target is found.

Store result = 2.

Continue searching the left half.

New right = 1.

Step 2:

Left = 0

Right = 1

Middle = (0 + 1) // 2 = 0

Timestamp at index 0 = 101

Since 101 is smaller than 102, search the right half.

New left = 1.

Step 3:

Left = 1

Right = 1

Middle = (1 + 1) // 2 = 1

Timestamp at index 1 = 102

The target is found again.

Update result = 1.

Continue searching the left half.

New right = 0.

Now left = 1 and right = 0, so the search stops.

Final Result:

First occurrence of timestamp 102 = index 1.

The corresponding order is:

(102, O2)

Therefore, the modified Binary Search correctly returns the first order with the given timestamp.

10.7 RECURRENCE RELATION OF BINARY SEARCH

Binary Search reduces the search range by half in every step.

The recurrence relation is:

$T(n) = T(n/2) + c$

Where:

- T(n) represents the running time.
- T(n/2) represents the remaining search operation.
- c represents the constant-time comparison and calculation.

Base condition:

$T(1) = c$

10.8 SOLUTION OF RECURRENCE

Expanding the recurrence:

$T(n) = T(n/2) + c$

$T(n) = T(n/4) + 2c$

$T(n) = T(n/8) + 3c$

After k steps:

$T(n) = T(n/2^k) + kc$

The search ends when:

$n/2^k = 1$

Therefore:

$n = 2^k$

Taking logarithm on both sides:

$k = \log_2 n$

Substituting:

$T(n) = T(1) + c \log_2 n$

Therefore:

$T(n) = \Theta(\log n)$

The time complexity of Binary Search is O(log n).

10.9 MAXIMUM COMPARISONS FOR 10,000,000 ORDERS

The KartIQ platform stores approximately 10,000,000 orders.

Let:

$n = 10,000,000$

The maximum number of iterations required by Binary Search is approximately:

$\lfloor \log_2 n \rfloor + 1$

$\log_2(10,000,000) \approx 23.2535$

Therefore:

Maximum iterations = 24

Hence, at most 24 middle-element checks are required in a standard Binary Search over 10,000,000 elements.

The modified first-occurrence search also takes O(log n) time and requires at most about 24 iterations for this dataset size.

10.10 ADVANTAGES OF BINARY SEARCH

1. It searches large datasets efficiently.
2. It requires fewer comparisons than Linear Search.

3. It reduces the search range by half in every iteration.
4. It is suitable for searching millions of orders.
5. The modified version can find the first occurrence of duplicate timestamps.

10.11 CONCLUSION OF MODULE 3

Binary Search is the most suitable algorithm for order lookup in KartIQ because it provides O(log n) time complexity.

The sorted order list generated by Merge Sort makes Binary Search possible.

The modified Binary Search ensures that when multiple orders share the same timestamp, the first order is returned correctly.

PAGE 11 – STRASSEN'S RECURRENCE AND COMPLEXITY ANALYSIS

11.10 RECURRENCE RELATION OF STRASSEN'S METHOD

Strassen's method divides each n × n matrix into four smaller matrices of size n/2 × n/2.

It performs seven recursive matrix multiplications and additional matrix additions and subtractions.

The recurrence relation is:

$T(n) = 7T(n/2) + cn^2$

Where:

- T(n) represents the running time.
- 7T(n/2) represents seven recursive matrix multiplications.
- cn^2 represents the addition and subtraction operations.
- n represents the dimension of the matrix.
- c is a constant.

Base condition:

$T(1) = c$

11.11 SOLUTION USING MASTER THEOREM

The general recurrence form is:

$T(n) = aT(n/b) + f(n)$

For Strassen's method:

$a = 7$

b = 2

f(n) = cn^2

Calculate:

n^(log_b a) = n^($\log_2 7$)

$\log_2 7 \approx 2.807$

Therefore:

n^($\log_2 7$) ≈ $n^{2.807}$

Since n^2 grows slower than $n^{2.807}$, the recurrence follows Case 1 of the Master Theorem.

Therefore:

T(n) = Θ(n^2.807)

Hence, Strassen's method has approximately O(n^2.807) time complexity.

11.12 CONVENTIONAL MATRIX MULTIPLICATION

Conventional matrix multiplication uses three nested loops.

For two n × n matrices, each element of the result matrix requires n multiplications.

There are n^2 elements in the result matrix.

Therefore, the total number of scalar multiplications is:

n × n × n = n^3

Time complexity:

T(n) = O(n^3)

11.13 COMPARISON FOR n = 1024

In the KartIQ platform:

n = 1024

Since 1024 = 2^{10}, it is suitable for repeated division into equal halves.

Conventional Matrix Multiplication:

Number of scalar multiplications = n^3

= 1024^3

= 1,073,741,824

Therefore, conventional matrix multiplication requires:

1,073,741,824 scalar multiplications.

Strassen's Method:

For n = 1024, there are 10 recursive levels because:

$1024 = 2^{10}$

At every level, the number of recursive multiplications is multiplied by 7.

Therefore:

Number of scalar multiplications = 7^{10}

= 282,475,249

Therefore, Strassen's method requires:

282,475,249 scalar multiplications.

11.14 COMPARISON TABLE

Method	Time Complexity	Scalar Multiplications for n = 1024
Conventional Matrix Multiplication	$O(n^3)$	1,073,741,824
Strassen's Method	O(n^2.807)	282,475,249

The difference is:

1,073,741,824 - 282,475,249

= 791,266,575

Therefore, Strassen's method saves approximately 791 million scalar multiplications compared with the conventional method under this recursive operation-count model.

11.15 WHY STRASSEN'S METHOD MAY NOT BE FASTER IN PRACTICE

Although Strassen's method reduces the number of multiplications, it may not always be faster in real-world applications.

The reasons are:

1. **Additional Additions:** Strassen's method performs more matrix additions and subtractions.
2. **Memory Usage:** Temporary matrices require additional memory.
3. **Implementation Complexity:** The algorithm is more complex than conventional matrix multiplication.
4. **Small Matrix Size:** For small matrices, conventional multiplication may be faster.
5. **Hardware Optimisation:** Conventional matrix multiplication is highly optimised in modern libraries and processors.

6. **Cache Performance:** Temporary matrix operations can affect memory access efficiency.

Therefore, the theoretical reduction in multiplication operations does not always guarantee better practical execution time.

11.16 PRACTICAL HYBRID APPROACH

A practical solution is to use a hybrid matrix multiplication algorithm.

The hybrid approach works as follows:

1. Start with Strassen's recursive multiplication for large matrices.
2. Divide the matrices into smaller submatrices.
3. Continue using Strassen's method while the matrix size is above a selected threshold.
4. When the matrix size becomes small, switch to conventional matrix multiplication.
5. Combine the results to produce the final matrix.

For example, Strassen's method can be used until the submatrix size reaches a selected threshold such as 64 × 64, and conventional multiplication can be used below that threshold.

The exact threshold should be selected through benchmarking on the target hardware.

If matrix dimensions are not powers of two, the matrices can be padded with zeros to suitable dimensions or handled using an appropriate block-based implementation.

11.17 CONCLUSION OF MODULE 4

Strassen's Matrix Multiplication is suitable for large recommendation matrices because it reduces the number of recursive multiplications from eight to seven.

Its time complexity is approximately O(n^2.807), compared with $O(n^3)$ for conventional multiplication.

However, additional memory and matrix addition operations must be considered. Therefore, a hybrid approach provides a practical solution for the KartIQ recommendation engine.

PAGE 12 – (f) DESIGN AND JUSTIFY THE FINAL SOLUTION

12. FINAL ALGORITHMIC DESIGN FOR KARTIQ

After analysing all four modules, suitable Divide and Conquer algorithms are selected to improve the performance of the KartIQ retail data analytics platform.

The final solution uses Merge Sort, Improved Quick Sort, Binary Search, and Strassen's Matrix Multiplication.

Each algorithm is selected based on the computational requirements, input size, time complexity, and memory requirements.

12.1 MODULE 1 – NIGHTLY ORDER CONSOLIDATION

Selected Algorithm: Stable Merge Sort

Merge Sort is selected to sort 10,000,000 orders according to their timestamps.

The algorithm divides the order list into smaller parts, sorts each part, and merges the sorted parts.

It maintains the original arrival order of orders having equal timestamps.

Time Complexity:

Best Case = O(n log n)

Average Case = O(n log n)

Worst Case = O(n log n)

Extra Memory:

O(n)

Reason for Selection:

1. It provides consistent performance for large datasets.
2. It maintains stability.
3. It is suitable for sorting millions of orders.
4. It guarantees O(n log n) time complexity.
5. External Merge Sort can handle data that does not fit into RAM.

Main Trade-off:

Merge Sort requires additional memory for temporary arrays.

Condition to Change the Algorithm:

If the complete dataset cannot fit into RAM, External Merge Sort should be used instead of an in-memory implementation.

12.2 MODULE 2 – LIVE SELLER LEADERBOARD

Selected Algorithm: Improved Quick Sort with Introsort Protection

Improved Quick Sort is selected for sorting the scores of 100,000 sellers.

Median-of-three or randomized pivot selection helps avoid repeatedly choosing a poor pivot.

For stronger worst-case protection, Introsort can switch to Heap Sort when recursion becomes too deep.

Time Complexity:

Best Case = O(n log n)

Average Case = O(n log n)

Worst Case with Introsort Protection = O(n log n)

Extra Memory:

O(log n) for typical recursive sorting, with implementation-dependent stack usage.

Reason for Selection:

1. It provides good average-case performance.
2. Better pivot selection reduces unbalanced partitions.
3. Introsort protects against $O(n^2)$ worst-case behaviour.
4. It is suitable for large seller leaderboards.
5. Small partitions can be handled using Insertion Sort.

Main Trade-off:

Quick Sort requires careful pivot selection and recursion management. Introsort is more complex to implement.

Condition to Change the Algorithm:

If the leaderboard is updated only slightly between refreshes, an incremental update strategy or an adaptive sorting method may be more efficient than sorting the entire list repeatedly.

12.3 MODULE 3 – ORDER LOOKUP

Selected Algorithm: Modified Binary Search

Binary Search is selected to find orders using their timestamps in the sorted list produced by Module 1.

The modified version returns the first occurrence when multiple orders have the same timestamp.

Time Complexity:

Best Case = O(1)

Average Case = O(log n)

Worst Case = O(log n)

Extra Memory:

O(1) for an iterative implementation.

Reason for Selection:

1. It works efficiently on sorted data.
2. It reduces the search range by half.
3. It requires very few comparisons.
4. It can handle millions of orders.
5. The modified version finds the first occurrence of duplicate timestamps.

Main Trade-off:

Binary Search requires the data to be sorted before searching.

Condition to Change the Algorithm:

If orders are frequently inserted into an unsorted data structure, a suitable indexed search structure or database index should be considered.

MODULE 4 – RECOMMENDATION SCORES

Selected Algorithm: Hybrid Strassen's Matrix Multiplication

Strassen's method is selected to calculate recommendation scores by multiplying two 1024 × 1024 matrices.

It reduces the number of recursive matrix multiplications from eight to seven.

For practical execution, a hybrid approach switches to conventional multiplication when the submatrices become small.

Time Complexity:

Strassen's Method = O(n^2.807)

Conventional Matrix Multiplication = $O(n^3)$

Extra Memory:

$O(n^2)$ additional temporary storage in a typical recursive implementation, depending on memory management and implementation details.

Reason for Selection:

1. It reduces the number of recursive multiplications.
2. It is suitable for large matrices.
3. It improves the theoretical time complexity.
4. It can reduce recommendation calculation time.
5. A hybrid implementation can reduce overhead for smaller matrices.

Main Trade-off:

Strassen's method requires additional additions, subtractions, and temporary memory.

Condition to Change the Algorithm:

13 – FINAL ALGORITHM COMPARISON

12.5 FINAL DESIGN COMPARISON TABLE

Module	Selected Algorithm	Time Complexity	Extra Memory	Main Reason
Nightly Order Consolidation	Stable Merge Sort	O(n log n)	O(n)	Stable sorting of large datasets
Live Seller Leaderboard	Improved Quick Sort with Introsort Protection	O(n log n) worst case	O(log n) typical stack usage	Better pivot selection and worst-case protection
Order Lookup	Modified Binary Search	O(log n)	O(1)	Fast searching and first occurrence
Recommendation Scores	Hybrid Strassen's Method	O(n^2.807) for Strassen-dominated execution	$O(n^2)$ typical temporary storage	Fewer recursive multiplications

12.6 OVERALL PERFORMANCE IMPROVEMENTS

The proposed algorithms provide improvements in the four major modules.

1. Faster Nightly Order Processing

Merge Sort processes large order datasets with O(n log n) time complexity. Its stability ensures that orders having equal timestamps remain in their original arrival order.

2. Improved Seller Leaderboard Performance

Improved pivot selection reduces the possibility of unbalanced partitions. Introsort protection prevents the algorithm from suffering the $O(n^2)$ worst-case behaviour of ordinary Quick Sort.

3. Faster Order Searching

Modified Binary Search can locate the first order having a particular timestamp in logarithmic time.

For 10,000,000 orders, it requires at most approximately 24 iterations.

4. Faster Recommendation Calculations

Strassen's method reduces the number of recursive matrix multiplications compared with conventional matrix multiplication.

A hybrid implementation balances theoretical efficiency with practical execution performance.

12.7 IMPORTANT DESIGN CONSIDERATIONS

While implementing the final algorithms, KartIQ must consider the following points:

1. All order records must preserve their original arrival sequence when timestamps are equal.
2. The seller leaderboard must avoid poor pivot selection.
3. Binary Search must only operate on data sorted by timestamp.
4. Duplicate timestamps must be handled correctly.
5. Matrix dimensions must be handled correctly during recursive multiplication.
6. Memory usage must be monitored for large datasets.
7. Performance must be measured using actual production-like input data.

12.8 OVERALL JUSTIFICATION

The selected algorithms are suitable because each one solves a different computational problem.

Merge Sort provides stable sorting, Improved Quick Sort provides efficient repeated sorting, Binary Search provides fast order lookup, and Strassen's method reduces the multiplication workload for large recommendation matrices.

The combination of these algorithms improves the efficiency and reliability of the KartIQ platform.

14 – OVERALL CONCLUSION

13. CONCLUSION

This case study analysed the performance problems of the KartIQ retail data analytics platform and proposed suitable Divide and Conquer algorithms for its four major modules.

The first module used Merge Sort for nightly order consolidation. The algorithm sorted the sample orders according to their timestamps and maintained the original arrival order of orders having equal timestamps. Its time complexity was derived as $O(n \log n)$.

The second module analysed Quick Sort for the live seller leaderboard. It showed that selecting the first element as the pivot can produce highly unbalanced partitions and $O(n^2)$ worst-case performance. Median-of-three pivot selection and Introsort protection were proposed to improve performance.

The third module applied Binary Search to order lookup. The recurrence was solved to obtain O(log n) time complexity. A modified Binary Search was also designed to return the first occurrence of a timestamp when duplicate values are present.

The fourth module analysed Strassen's Matrix Multiplication for recommendation score calculation. The seven multiplication formulas were derived and verified using the sample matrices. The recurrence was solved to obtain O(n^2.807) time complexity. A hybrid approach was proposed to improve practical performance.

Finally, the four algorithms were compared based on their time complexity, memory requirements, advantages, and trade-offs.

The study demonstrates that choosing suitable algorithms is essential for handling large-scale retail data. By applying these Divide and Conquer techniques, KartIQ can improve order processing, seller ranking, order searching, and product recommendation calculations.

Therefore, the proposed algorithmic design provides an efficient and scalable solution for the KartIQ retail data analytics platform.

14. REFERENCES

1. Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, *Introduction to Algorithms*, MIT Press.
2. Ellis Horowitz, Sartaj Sahni, and Sanguthevar Rajasekaran, *Computer Algorithms*.
3. Mark Allen Weiss, *Data Structures and Algorithm Analysis*.
4. Course notes and classroom materials for CSA06 – Design and Analysis of Algorithms.
5. Standard algorithm analysis concepts related to Merge Sort, Quick Sort, Binary Search, and Strassen's Matrix Multiplication.